

# 算法驱动的迷宫探险游戏开发方法

张佳栋 方维 逯熙钰 姜舜航

(东北大学计算机科学与工程学院 沈阳 110169)

**摘 要** 本文提出了一种融合经典算法策略的迷宫探险游戏开发方法, 涵盖从迷宫生成到智能策略执行的完整流程。首先, 基于分治法生成满足“无孤立区域存在唯一通路”的随机迷宫, 支持多种尺寸, 并实现了游戏过程的 OpenGL 动态可视化。其次, 采用树形动态规划实现全局资源收集路径规划, 确保在避开陷阱的前提下最大化资源总值; 实验中, 该方法在 100 组  $7 \times 7$  迷宫中的平均资源得分为 6.72。针对有限视野场景, 设计了一种贪心策略和启发式策略, 通过预期收益函数实现局部最优选择, 平均得分达 6.18, 较普通贪心策略 (4.89) 提高 26.3%。此外, 利用回溯法构建密码解谜系统, 在限制条件下成功解出所有 3 位数谜题; 在 BOSS 战斗模块中, 基于分支限界法实现技能序列优化策略, 在可用资源有限的情况下稳定输出最小回合击败序列。整体框架展示了各类算法在路径规划、决策优化和策略推理中的有效结合, 为算法驱动的游戏开发提供了实用范式。

**关键词** 迷宫生成; 树形 DP; 贪心策略; 启发式策略; 回溯法; 分支限界法; 可视化  
**中图法分类号** TP

## Algorithm-Driven Maze Exploration Game Development Method

ZHANG Jia-Dong FANG Wei LU Xi-Yu JIANG Shun-Hang

(School of Computer Science and Engineering, Northeastern University, Shenyang 110169, China)

**Abstract** This paper presents a maze exploration game development framework integrating classical algorithmic strategies, covering the entire process from maze generation to intelligent strategy execution. First, a divide-and-conquer approach is used to generate random mazes that guarantee unique solvable paths without isolated regions. The system supports multiple maze sizes and features real-time OpenGL-based visualization. Second, a tree-based dynamic programming method is adopted for global resource collection path planning, ensuring maximum total value while avoiding traps. In experiments on 100 randomly generated  $7 \times 7$  mazes, this method achieved an average resource score of 6.72%. For limited-vision scenarios, both a greedy strategy and a heuristic strategy are designed. A reward expectation function guides local optimal selection, resulting in an average score of 6.18 with an improvement of 26.3% over the basic greedy approach (4.89). Additionally, a backtracking algorithm is employed to construct a password puzzle system, successfully solving all 3-digit puzzles under given constraints. In the boss battle module, a branch-and-bound method is used to optimize the skill sequence, consistently generating minimal-turn victory strategies under limited player resources. This framework demonstrates an effective integration of various algorithms in path planning, decision-making, and strategic reasoning, offering a practical paradigm for algorithm-driven game development.

**Keywords** maze generation; tree-based DP; greedy strategy; heuristic strategy; backtracking; branch and bound; visualization

## 1 项目概述

本项目旨在开发一款融合经典算法与图形可视化的迷宫探险游戏. 通过将分治、动态规划、贪心、回溯、分支限界等算法设计策略贯穿于迷宫生成、资源收集、谜题解锁、战斗系统等多个关键模块, 实现算法理论与工程实践的深度融合.

项目主要有以下模块:

- (1) 分治法生成随机迷宫
- (2) 动态规划求解最优资源收集路径
- (3) 贪心算法实时策略模拟
- (4) 启发式算法实时策略模拟
- (5) 回溯法解锁机关密码
- (6) 分支限界法优化 BOSS 战斗策略
- (7) 基于 OpenGL 的可视化演示

## 2 分治法地图生成

### 2.1 问题描述

任务: 使用分治法生成迷宫, 无孤立区域, 存在唯一通路.

输入: 迷宫尺寸.

输出: 迷宫矩阵 (可存储为 JSON 或 CSV), 包括起点 Start(S)、终点 Exit(E)、墙壁 (#)、通路 (空格)、资源 (例如金币 G)、陷阱 Trap(T)、机关 Locker(L)、BOSS(B).

### 2.2 算法设计

#### 1. 初始化迷宫

创建二维数组表示迷宫, 尺寸为奇数 (如  $height \times width$ ).

#### 2. 递归分割区域

输入参数: 子问题迷宫的左上方坐标和右下方坐标.

终止条件: 区域宽度或高度小于 2.

#### 3. 选择分割方向

随机选择一个纵向墙体(非外围) 和一个横向墙体(非外围), 视其把迷宫分割为四个部分作为其子问题并求解, 同时在四片墙体中随机选取三个墙体将其打通, 由于  $2 \times 2$  迷宫的特殊性, 这样生成的迷宫路径唯一且无环路, 且无需合并子问题.

#### 4. 设置入口和出口

入口: 在迷宫最外围墙体(非四个角) 上随机选择一个作为入口.

出口: 在迷宫最外围墙体(非四个角) 上随机选择一个作为出口, 若与出口重合, 则重新生成直到不同.

#### 5. 添加资源和陷阱

随机在迷宫中添加资源和陷阱, 数量和位置可根据需求设定.

### 2.3 复杂度分析

下面用归纳法证明对于生成一个  $a \times b$  的迷宫的时间复杂度是  $O(a \times b)$ .

由于最小子问题是一个  $1 \times n$  或  $n \times 1$  大小的块, 处理这样一个子问题的复杂度是  $O(n)$  的, 满足归纳假设.

假设对于所有  $a < A, b < B$  的问题的复杂度  $T(a, b) = O(a \times b)$ .

对于一个  $A \times B$  的问题, 假设划分大小为  $p, q$ . 划分的复杂度为  $O(1)$ .

那么复杂度

$$\begin{aligned} T(A, B) &= O(1) + T(p, q) + T(p, B - q) + \\ &\quad + T(A - p, q) + T(A - p, B - q) \\ &= O(1) + O(p \times q) + O(p \times (B - q)) + \\ &\quad + O((A - p) \times q) + O((A - p) \times (B - q)) \\ &= O(A \times B) \end{aligned}$$

所以, 规模为  $a \times b$  的问题的时间复杂度是  $O(a \times b)$ .

特别的, 当  $a = b = n$  时, 复杂度为  $O(n^2)$ .

### 2.4 伪代码

见附录 1.

## 3 动态规划求解最优路径

### 3.1 问题描述

任务: 计算从起点到终点的最优资源收集路径, 避开陷阱, 优先拾取资源. 例如: 状态  $dp[i][j]$  表示走到坐标  $(i, j)$  时的最大资源值.

输入: 迷宫矩阵、资源分布(例如金币 = 5)、陷阱位置(可设置陷阱 = -3).

输出: 最大资源值、最优路径序列.

需求: 路径可视化(可选).

### 3.2 问题分析

目标: 从迷宫的起点出发, 到达终点, 经过必须经过的机关和 BOSS, 收集最多的资源.

分析: 地图保证了路径唯一, 可知地图实际是一个树形结构, 可以采用树形 DP. 我们以起点为根, 构建地图对应的树. 问题就转化为了有一颗带权树, 要求找一条从根到终点的通路, 使得通路上的权值和最大.

### 3.3 状态设计及状态转移方程

定义  $dp_{i,j}$  表示从点  $(i, j)$  出发能获得的最大价值,  $must_{i,j}$  表示点  $(i, j)$  是否为从起点到终点必须经过的点. 那么答案即为  $dp_s$ , 其中  $s$  为起点的坐标. 那么有转移方程:

$$\text{must}_p = \begin{cases} \bigvee_{x \in \text{son}_p} \text{must}_x, & \text{if } x \text{ is not E} \\ 1, & \text{if } x = \text{is E} \end{cases}$$

$$\text{dp}_p = w_p + \sum_{x \in \text{son}_p} \begin{cases} \text{dp}_x, & \text{if } \text{must}_x \\ \max(\text{dp}_x, 0), & \text{else} \end{cases}$$

### 3.4 最优路径构建

对于不同  $\text{must}$  的点的构建方法不同.

对于  $\text{must}$  为真的点, 需要先输出自己的坐标, 然后依次构建  $\text{must}$  为假并且采用其中权值的子树的路径, 中间穿插自己的坐标, 最后构建  $\text{must}$  为真的子树的路径.

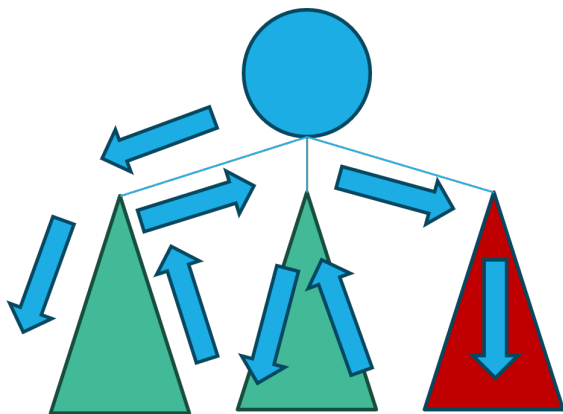


Fig. 1  $\text{must}$  为真的点的结构



Fig. 2  $\text{must}$  为真的点的路径构建顺序

### 3.5 伪代码

见附录 1.

### 3.6 二级标题\*字体为5号黑体\*标题2

#### 3.6.1 三级标题\*字体为5号宋体\*标题3

\*正文部分, 字体为5号宋体\* 正文文字

正文文字要求语句通顺, 无语法错误, 结构合理, 条理清楚, 不影响审稿人、读者阅读理解全文内容。以下几类问题请作者们特别注意:

- 1) 文章题目应明确反映文章的思想和方法; 文字流畅, 表述清楚;
- 2) 中文文字、英文表达无语法错误;
- 3) 公式中无符号、表达式的疏漏, 没有同一个符号表示两种意思的情况;
- 4) 数学中使用的符号、函数名用斜体;
- 5) 使用的量符合法定计量单位标准;
- 6) 矢量为黑体, 标量为白体;

7) 变量或表示变化的量用斜体;

8) 图表规范, 量、线、序无误, 位置正确(图表必须在正文中有所表述后出现, 即...如图1所示)(注意纵、横坐标应有坐标名称和刻度值)。

9) 列出的参考文献必须在文中按顺序引用, 即参考文献顺序与引用顺序一致, 各项信息齐全(格式见参考文献部分);

10) 首次出现的缩写需写明全称, 首次出现的符号需作出解释。

11) 图的图例说明、坐标说明全部用中文或量符号。

12) 图应为矢量图。

13) 表中表头文字采用中文。

14) 公式尺寸:

标准: 10.5磅

下标/上标: 5.8磅

次下标/上标: 4.5磅

符号: 16磅

次符号: 10.5磅

15) 组合单位采用标准格式, 如: “pJ/bit/m<sup>4</sup>”应为“pJ/(bit·m<sup>4</sup>)”

**定理1.** \*\*\*\*\*. \*定理内容.\*

["定义"、“假设”、“公理”、“引理”等的排版格式与此相同, 详细定理证明、公式可放在附录中]

证明. \*证明过程.\* ["例x"等的排版格式相同] 证毕.

#### 示例图片

(请插入当时做图时的矢量版 如有当时的文件, 例如 Visio, origin, matlab, smartdraw, Excel, powerpoint 等各种软件作的图, 图字用6号宋体, 外文Times new roman, 图中文字尽量用翻译成中文)

如插入图为截图, 必将原作图文件如\*.vsd, \*.opj, \*.fig, \*.sdr, \*.eps, \*.emf, \*.wmf, \*.ps 等后缀名随修改稿压缩后传过来排版。

图X 图片说明\*字体为小5号, 图片应为黑白图, 图中的子图要有子图说明\*

#### 表X 表说明\*表说明采用黑体\*

\*示例表格\* \*第1行为表头, 表头要有内容\*

**过程X. 过程名称**

\*《计算机学报》的方法过程描述字体为小5号宋体, IF、THEN等伪代码关键词全部用大写字母, 变量和函数名称用斜体\*

**算法Y. 算法名称.**

输入: ... ..

输出: ... ..

\*《计算机学报》的算法描述字体为小5号宋体, IF、THEN等伪代码关键词全部用大写字母, 变量和函数名称用斜体\*

**致 谢** \*致谢内容.\* 致谢

**参 考 文 献**

- [1] 网上的文献(举例: The Cooperative Association for Internet Data Analysis(CAIDA),<http://www.caida.org/data> 2010,7,18) \*请采用脚注放于正文出现处, 每页的脚注从1开始编号\*
- [2] 中文的参考文献需给出中英文对照。形式如[3]。
- [3] Zhou Yong-Bin, Feng Deng-Guo. Design and analysis of cryptographic protocols for RFID. Chinese Journal of

Computers, 2006, 29(4): 581-589 (in Chinese)

(周永彬, 冯登国. RFID安全协议的设计与分析. 计算机学报, 2006, 29(4): 581-589)

- [4] 期刊、会议、书籍名称不能用缩写。
- [5] 作者(外国人姓在前, 名在后可缩写, 后同). 题目(英文题目第一字母大写, 其它均小写): 副标题(如果有). 刊名(全称), 年, 卷(期): 页码\*期刊论文格式\*
- [6] 作者. 文章题目(英文题目第1字母大写, 其它均小写): 副标题(如果有)//Proceedings of the ... (会议名称). 会议召开城市, 会议召开城市所在国家, 年: 页码\*会议论文集论文格式\*
- [7] 作者. 文章题目(英文题目第一字母大写, 其它均小写): 副标题(如果有)//编者. 文集标题. 出版地: 出版社, 出版年: 页码\*文集格式\*
- [8] 作者. 书名: 副标题(如果有). 版次(初版不写). 出版社地点: 出版社, 出版年\*书籍格式\*
- [9] 作者. 文章题目[博士学位论文/硕士学位论文]. 单位名称, 单位地点, 年\*学位论文格式\*
- [10] 作者. 文章题目(英文题目第一字母大写, 其它均小写). 单位地点: 单位, 技术报告: 报告编号, 年\*技术报告\*
- [11] 专利拥有人. 专利名称, 专利授权国家, 专利授权日期\*技术专利\*

**附录1. 伪代码**

**Algorithm 1** Maze Generation

---

```

1: function GENMAZE( $n$ )
2:   Initialize maze  $M$  with walls
3:   Set entrance and exit on  $M$  boundary
4:    $P \leftarrow \emptyset$ 
5:    $r \leftarrow \frac{n+1}{2}$ 
6:   GETPATH( $P, 0, 0, r-2, r-2$ )
7:   DESTROYWALL( $P, M$ )
8:   GENERATEELEMENTS( $M$ )
9:   return  $M$ 
10: end function
11: function GETPATH( $P, x_1, y_1, x_2, y_2$ )
12:   if  $x_1 = x_2$  then
13:     for  $i = y_1$  to  $y_2 - 1$  do
14:        $P \leftarrow P \cup \{((x_1, i), (x_1, i+1))\}$ 
15:     end for
16:     return
17:   end if
18:   if  $y_1 = y_2$  then
19:     for  $i = x_1$  to  $x_2 - 1$  do
20:        $P \leftarrow P \cup \{((i, y_1), (i+1, y_1))\}$ 
21:     end for
22:     return
23:   end if
24:   Choose random  $d_i \in [x_1, x_2 - 1], d_j \in [y_1, y_2 - 1]$ 
25:   Open doors around  $(d_i, d_j)$ , add to  $P$ 
26:   GETPATH( $P, x_1, y_1, d_i, d_j$ )
27:   GETPATH( $P, d_i + 1, y_1, x_2, d_j$ )
28:   GETPATH( $P, d_i + 1, d_j + 1, x_2, y_2$ )
29:   GETPATH( $P, x_1, d_j + 1, d_i, y_2$ )
30: end function
31: function DESTROYWALL( $P, M$ )
32:   for all  $e = ((x_1, y_1), (x_2, y_2)) \in P$  do
33:     if  $x_1 = x_2$  then
34:       Remove vertical wall between points in
35:        $M$ 
36:     else
37:       Remove horizontal wall between points
38:       in  $M$ 
39:     end if
40:   end for
41: end function
42: function GENERATEELEMENTS( $M$ )
43:   for all empty cell  $c$  do
44:     Place boss, locker, trap, resource with given
45:     probabilities
46:   end for
47: end function

```

---